d_01: BPOエージェントエンジン設計 — ナレッジからAI社員を生む

> **位置づけ**: 本書はd_04 Agent OS内のゲノム駆動エンジン詳細設計に該当する。上位設計はd_04を参照。
>
> **本質**: テンプレート/ナレッジに入った業務知識を「AI社員」に変換し、
> 自律的にBPOタスクを発見・実行・学習するエンジンを設計する。
> 海外AI社員プロダクト（11x/Lindy/Salesforce Agentforce等）の共通パターンを
> シャチョツーのナレッジ基盤の上に実装する。

戦略変更（2026-03-27）: ゲノム駆動型アーキテクチャへの転換

> **従来**: 1業種=1ディレクトリ（16業種×個別パイプライン = 80+ファイル）
> **新方針**: 共通エンジン + ゲノムJSON で全業種対応。新業種追加 = JSON 1ファイル追加のみ。

コア6業界（Phase 1 集中対象）

1. 建設業（47万社）
2. 製造業（38万社）
3. 医療・福祉（25万施設）
4. 不動産業（35万社）
5. 運輸・物流（7万社）
6. 卸売業（25万社）— 新規追加

ゲノム駆動の設計原則

```
workers/bpo/
├── engine/          ← 共通パイプラインエンジン（5-6本）
│   ├── ocr_extract.py      # OCR→抽出（全業種共通）
│   ├── document_check.py   # 書類照合（全業種共通）
│   ├── estimation.py       # 見積・計算（全業種共通）
│   ├── report_generate.py  # レポート生成（全業種共通）
│   └── approval_flow.py    # 承認フロー（全業種共通）
│
├── genome/data/
│   ├── construction.json   # 建設の業務定義・用語・ルール
│   ├── manufacturing.json  # 製造の業務定義・用語・ルール
│   ├── wholesale.json      # 卸売 = JSONを1個足すだけ
│   └── ...                 # 100業種でもJSON追加だけ
│
└── micro/           ← 原子操作（業種に依存しない）
```

子会社モデルとの接続

- 本体 = 共通エンジン + インフラ + AI進化（プラットフォーム利用料30%）
- 子会社 = ゲノムJSON定義 + 業界営業 + ドメイン知識（売上の70%）
- Phase 2でゲノムオーサリングUI（業界パートナーがブラウザでBPOフロー定義）

---

0. 全体アーキテクチャ

```
┌──────────────────────────────────────────────────────────────────────┐
│                        BPO Agent Engine                              │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  [A] Agent Factory          [B] Task Discovery        [C] Executor   │
│  ─────────────────          ──────────────────        ─────────────  │
│  ナレッジ → AI社員定義       スケジュール/イベント      承認 → 実行     │
│  role + tools + rules        → タスク自動発見           → 結果記録     │
│                                                                      │
│  [D] Feedback Loop          [E] Trust Scorer          [F] Audit Card │
│  ─────────────────          ──────────────────        ─────────────  │
│  却下理由 → ナレッジ更新     信頼度スコア計算           AI行動の可視化  │
│  承認 → 精度向上             → 自律度レベル昇格                        │
│                                                                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ┌──────────┐  ┌──────────────┐  ┌────────────┐  ┌──────────────┐  │
│  │ knowledge │  │ knowledge    │  │ tool       │  │ execution    │  │
│  │ _items    │  │ _relations   │  │ _connections│  │ _logs        │  │
│  │ (Brain)   │  │ (triggers)   │  │ (SaaS API) │  │ (BPO結果)    │  │
│  └──────────┘  └──────────────┘  └────────────┘  └──────────────┘  │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

A. Agent Factory — ナレッジからAI社員を自動生成

A.1 概念: knowledge_items → BPO Agent Role

海外AI社員プロダクトの「Job Description型」ロール定義を、
ナレッジのメタデータから**自動生成**する。

```
# 例: common.json の「経費精算フロー」アイテムから生成されるAI社員

BPOAgentRole:
  id: "agent_expense_processor"
  name: "経費精算AI社員"
  department: "経理・財務"

  # ← knowledge_items.content から自動抽出
  job_description: |
    経費精算の申請を受け付け、ルールに基づいて自動処理する。
    - 領収書OCR → 仕訳候補生成 → 承認WF起動
    - インボイス要件チェック（登録番号・税率区分）
    - 電帳法要件チェック（タイムスタンプ・検索要件）

  # ← knowledge_items.conditions + exceptions から抽出
  rules:
    - "{{expense_receipt_threshold}}円以上は領収書必須"
    - "交通費は公共交通機関の実費精算が原則"
    - "タクシー利用は事前承認制"
    - "接待交際費は社長承認"

  # ← knowledge_items.bpo_method から抽出
  tools:
    - freee_expense_api      # 経費精算の読み書き
    - freee_journal_api      # 仕訳の下書き作成
    - ocr_receipt_reader     # レシートOCR
    - slack_notification     # 承認依頼の通知

  # ← knowledge_items.bpo_automatable + scale_trigger から
  initial_level: 0  # Level 0: 通知のみからスタート

  # ← knowledge_items.legal_basis から
  compliance_checks:
    - "電子帳簿保存法（2024年1月完全義務化）"
    - "インボイス制度（2023年10月〜）"
```

A.2 Agent Factory パイプライン

```
Step 1: ナレッジスキャン
━━━━━━━━━━━━━━━━━━━━━━━━
  knowledge_items から bpo_automatable = true のアイテムを抽出
  → 部門(department)ごとにグルーピング

Step 2: ロール合成
━━━━━━━━━━━━━━━━━━━━━━━━
  同一部門のアイテム群 → 1つのAI社員ロールに合成
  LLMで job_description を自然言語生成
  rules / tools / compliance_checks を構造化抽出

Step 3: ツールバインディング
━━━━━━━━━━━━━━━━━━━━━━━━
  bpo_method に記載のSaaS → tool_connections テーブルと照合
  接続済みSaaS → ツールとして有効化
  未接続SaaS → 「接続すると自動化できます」と提案

Step 4: トリガー発見
━━━━━━━━━━━━━━━━━━━━━━━━
  knowledge_relations の triggers チェーンを辿り、
  「いつ」「何をきっかけに」このAI社員が動くかを特定
  → スケジュールトリガー（月末、毎月10日、等）
  → イベントトリガー（SaaSデータ変更、Slack投稿、等）
  → 条件トリガー（残業45h超、有給未取得、等）

Step 5: Agent 登録
━━━━━━━━━━━━━━━━━━━━━━━━
  bpo_agents テーブルに登録
  初期 Level = 0（通知のみ）
  社長の承認で有効化
```

A.3 自動生成されるAI社員の例（common.json ベース）

AI社員名	部門	担当ナレッジ	ツール	トリガー
経費精算AI	経理	経費精算フロー + 電帳法 + インボイス	freee経費/OCR/Slack	申請イベント(随時)
請求・入金AI	経理	請求書発行 + 入金管理 + 消費税	freee請求/銀行API	月末締め(スケジュール)
月次決算AI	経理	月次決算フロー + 資金繰り	freee会計/Excel	月初5営業日(スケジュール)
給与計算AI	総務	給与計算 + 社保手続き + 源泉税	freee人事労務/SmartHR	毎月締め日(スケジュール)
勤怠管理AI	総務	勤怠管理 + 36協定 + 有給	ジョブカン/KING OF TIME	日次集計 + 月次レポート
採用・入退社AI	総務	採用フロー + 入退社手続き	SmartHR/求人媒体	入退社イベント(随時)
安全衛生AI	総務	健康診断 + ストレスチェック + 委員会	SmartHR/外部委託	年次スケジュール
契約管理AI	法務	契約管理 + 反社チェック	クラウドサイン/TDB	更新期限3ヶ月前アラート
セキュリティAI	情報	セキュリティ + IT資産 + バックアップ	ジョーシス/MDM	入退社イベント + 四半期棚卸

+ 業界固有AI社員（industry template から生成）：

AI社員名	業界	担当ナレッジ	ツール
安全書類AI	建設	安全管理 + KY活動 + 天候基準	ANDPAD/Photoruction
品質管理AI	製造	受入検査 + 工程内検査 + クレーム対応	kintone/ISO文書管理
予約・リコールAI	歯科	予約管理 + リコール + カウンセリング	Dentis/予約システム

B. Task Discovery — タスクの自動発見

B.1 3種類のトリガー

```
■ スケジュールトリガー（Cron型）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

  ナレッジの content から時間表現を抽出:
    「毎月{{attendance_closing_day}}日に締め」→ Cron: 0 9 25 * *
    「翌月5営業日以内に確定」→ Cron: 営業日計算
    「年1回の定期健康診断」→ Cron: 年次 + リマインド3ヶ月前
    「有給5日未取得チェック」→ Cron: 四半期ごと

  LLMで content → Cron式 を自動変換
  → bpo_schedules テーブルに登録

■ イベントトリガー（Webhook/Polling型）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  SaaS側のイベントを検知:
    freee: 新しい経費申請が作成された
    ジョブカン: 残業が月35時間を超えた
    Slack: #general に「請求書」を含む投稿
    SmartHR: 新しい入社者が登録された

  → Webhook or ポーリング(5分間隔) で検知
  → 該当AI社員にタスクをディスパッチ

■ 条件トリガー（knowledge_relations の triggers チェーン）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  ナレッジ間の因果関係を辿る:
    KI「残業が月45時間に到達」
      → triggers → KI「所属長が面談を実施」
      → triggers → KI「人事部に報告」

  あるKIの条件が満たされると、triggers先のKIに対応する
  AI社員が自動的にタスクを生成する
```

B.2 Task Discovery Engine（コアロジック）

```
class TaskDiscoveryEngine:
    """ナレッジからBPOタスクを自動発見するエンジン"""

    async def discover_tasks(self, company_id: str) -> list[BPOTask]:
        tasks = []

        # 1. スケジュールトリガー: 今日実行すべきタスクを取得
        scheduled = await self._check_schedules(company_id)
        tasks.extend(scheduled)

        # 2. イベントトリガー: SaaSイベントキューから未処理を取得
        events = await self._poll_saas_events(company_id)
        tasks.extend(events)

        # 3. 条件トリガー: knowledge_relationsのtriggersを評価
        triggered = await self._evaluate_triggers(company_id)
        tasks.extend(triggered)

        # 4. プロアクティブ発見: ナレッジの conditions を現在データと照合
        proactive = await self._proactive_scan(company_id)
        tasks.extend(proactive)

        return tasks

    async def _proactive_scan(self, company_id: str) -> list[BPOTask]:
        """
        ナレッジの conditions/examples を現在のSaaSデータと照合し、
        「今やるべきこと」を能動的に発見する。

        例:
          ナレッジ: 「有給取得が年5日未満の社員には個別に取得を促す」
          + ジョブカンAPI: 田中さんの有給取得 = 3日（残り9ヶ月で2日必要）
          → タスク生成: 「田中さんに有給取得を促すリマインド送信」
        """
        ...
```

C. Executor — レベル別実行エンジン

C.1 実行レベルと各AI社員の権限

```
Level 0: 通知（リスクゼロ）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  AI社員 → 「明日は月末請求の日です。先月は15件でした」
  ツール権限: READ のみ（SaaS GET API）
  出力: Slack/LINE通知 + ダッシュボード表示

Level 1: データ収集・レポート（読み取りのみ）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  AI社員 → 「kintoneから受注データを集計。15件 / ¥4,500,000。前月比+12%」
  ツール権限: READ のみ
  出力: 集計レポート + 前月比 + 異常値アラート

Level 2: 下書き作成（書くが確定しない）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  AI社員 → 「freeeに請求書の下書きを15件作成しました。確認してください」
  ツール権限: READ + WRITE:DRAFT
  出力: 下書き一覧 + 差分ハイライト + [確認][修正][却下] ボタン

  ★ ここが最も重要な価値。下書きなので間違っても被害なし。
  ★ 社長は「確認して承認するだけ」の世界。

Level 3: 承認付き実行（確定するが承認必要）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  AI社員 → 「請求書15件を確定して送付します。よろしいですか？」
  ツール権限: READ + WRITE:ALL（承認後のみ）
  出力: 計画表示 → [承認][修正][却下] → 実行 → 結果レポート
  昇格条件: Level 2で承認率 ≥ 90% + 社長明示承認

Level 4: 自律実行（信頼蓄積後）
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
  AI社員 → 「毎月の請求書処理を自動実行しました。レポートをご確認ください」
  ツール権限: READ + WRITE:ALL（自律）
  出力: 事後レポート + 異常時のみアラート
  昇格条件: Level 3で100回連続成功 + 成功率 ≥ 95% + 社長明示承認
  安全弁: いつでもLevel 3に戻せる
```

C.2 Executor のコア処理

```
class BPOExecutor:
    """レベルに応じてタスクを実行するエンジン"""

    async def execute(self, task: BPOTask, agent: BPOAgent) -> ExecutionResult:
        level = agent.current_level

        if level == 0:
            return await self._notify(task, agent)
        elif level == 1:
            return await self._collect_and_report(task, agent)
        elif level == 2:
            return await self._create_draft(task, agent)
        elif level == 3:
            approval = await self._request_approval(task, agent)
            if approval.status == "approved":
                return await self._execute_with_audit(task, agent)
            elif approval.status == "rejected":
                await self._learn_from_rejection(task, agent, approval)
                return ExecutionResult(status="rejected", reason=approval.comment)
        elif level == 4:
            result = await self._execute_with_audit(task, agent)
            await self._post_execution_report(task, agent, result)
            return result

    async def _create_draft(self, task: BPOTask, agent: BPOAgent) -> ExecutionResult:
        """
        Level 2: 下書き作成の核心ロジック

        1. ナレッジの rules を取得
        2. SaaSから現在データを取得
        3. LLMがルールとデータを照合して下書きを生成
        4. 差分をハイライトして承認UIに表示
        """
        # ナレッジからルールを取得
        rules = await self._get_agent_rules(agent)

        # SaaSからデータを取得（tool_connections経由）
        current_data = await self._fetch_saas_data(task, agent)

        # LLMがルール × データで下書きを生成
        draft = await self._llm_generate_draft(
            rules=rules,
            data=current_data,
            task=task,
            agent_role=agent.job_description,
        )

        # 下書きをSaaSに保存（draft status）
        draft_result = await self._save_draft_to_saas(draft, task, agent)
```

```
        # 承認リクエストを作成（監査カード付き）
        await self._create_approval_with_audit_card(
            task=task,
            agent=agent,
            draft=draft_result,
            reasoning=draft.reasoning,  # AI がなぜこう判断したかの説明
        )

        return ExecutionResult(status="draft_created", draft_id=draft_result.id)
```

D. Feedback Loop — 却下理由からナレッジを更新

D.1 学習パイプライン

```
■ 承認された場合
──────────────
  → trust_score を +1
  → 連続成功カウントを +1
  → 昇格条件を評価

■ 修正された場合（承認はするが内容を変更）
──────────────────────────────
  → 修正内容（diff）を記録
  → LLMが diff を分析:
      「社長は請求書の備考欄に工事番号を入れている」
    → 新ルール候補として提案:
        「請求書の備考欄には工事番号を記載する」
    → 社長が確認 → knowledge_items に追加
  → trust_score は変更なし（学習機会）

■ 却下された場合
──────────────
  → 却下理由（comment）を記録
  → LLMが理由を分析:
      「この業者への支払いは翌々月末にしている」
    → 既存ルールの例外として追加:
      exceptions: ["○○建設への支払いは翌々月末"]
    → knowledge_items を更新
  → trust_score を -2
  → 連続成功カウントをリセット

■ 定期学習（月次）
──────────────
  過去1ヶ月の全修正・却下を集約分析:
  → パターンを抽出
  → ナレッジの rules / examples / exceptions を更新提案
  → 社長が一括レビュー → 承認されたものを反映
```

D.2 ナレッジ更新のフロー

```
class FeedbackLearner:
    """却下・修正からナレッジを学習するエンジン"""

    async def learn_from_rejection(
        self, task: BPOTask, agent: BPOAgent, rejection: Approval
    ) -> list[KnowledgeUpdateProposal]:
        """
        却下理由を分析し、ナレッジの更新提案を生成する。

        Returns: 社長に確認してもらう更新提案のリスト
        """
        # 関連するナレッジアイテムを取得
        related_items = await self._get_related_knowledge(task, agent)

        # LLMで却下理由 × 既存ルール → 更新提案を生成
        proposals = await self._llm_analyze_rejection(
            rejection_reason=rejection.comment,
            task_details=task,
            existing_rules=related_items,
        )

        # 更新提案を proactive_proposals テーブルに保存
        for p in proposals:
            await self._save_proposal(
                company_id=agent.company_id,
                proposal_type="knowledge_update",  # rule_challenge / improvement
                source_agent=agent.id,
                target_knowledge_id=p.target_item_id,
                proposed_change=p.change,
                reasoning=p.reasoning,
            )

        return proposals
```

E. Trust Scorer — 信頼度スコアと自律度レベル管理

E.1 信頼度スコアの計算

```
class TrustScorer:
    """AI社員ごとの信頼度スコアを管理"""

    def calculate_score(self, agent: BPOAgent) -> TrustScore:
        return TrustScore(
            # 基本スコア（直近30日の実績）
            approval_rate=agent.approved_count / max(agent.total_count, 1),
            rejection_rate=agent.rejected_count / max(agent.total_count, 1),
            modification_rate=agent.modified_count / max(agent.total_count, 1),

            # 連続成功
            consecutive_successes=agent.consecutive_successes,

            # 重み付きスコア（却下は重い）
            weighted_score=(
                agent.approved_count * 1.0
                - agent.rejected_count * 3.0
                - agent.modified_count * 0.5
            ) / max(agent.total_count, 1),
        )

    def evaluate_level_up(self, agent: BPOAgent, score: TrustScore) -> LevelUpResult:
        """昇格条件の評価"""
        current = agent.current_level

        if current == 0 and agent.total_count >= 5:
            # Level 0→1: 5回以上の通知実行 + SaaS接続済み
            return LevelUpResult(eligible=True, next_level=1)

        elif current == 1 and score.approval_rate >= 0.8 and agent.total_count >= 20:
            # Level 1→2: 承認率80%以上 + 20回以上
            return LevelUpResult(eligible=True, next_level=2)

        elif current == 2 and score.approval_rate >= 0.9 and score.consecutive_successes
>= 30:
            # Level 2→3: 承認率90%以上 + 30回連続成功 + 社長明示承認
            return LevelUpResult(eligible=True, next_level=3,
requires_ceo_approval=True)

        elif current == 3 and score.approval_rate >= 0.95 and
score.consecutive_successes >= 100:
            # Level 3→4: 承認率95%以上 + 100回連続成功 + 社長明示承認
            return LevelUpResult(eligible=True, next_level=4,
requires_ceo_approval=True)
```

```
        return LevelUpResult(eligible=False)
```

F. Audit Card — AI行動の可視化（HubSpot Breeze型）

F.1 監査カードの構造

社長がAI社員の行動を「透明に」見られることが信頼の根幹。

```
┌──────────────────────────────────────────────┐
│ 📋 経費精算AI の実行レポート                    │
│ 2026-03-16 09:15                              │
├──────────────────────────────────────────────┤
│                                               │
│ ■ タスク: 3月分経費精算の処理（12件）           │
│                                               │
│ ■ AI の判断プロセス:                            │
│ 1. freeeから未処理の経費申請12件を取得          │
│ 2. 各申請をルールと照合:                        │
│    ✅ 8件: ルール通り → 仕訳下書き作成          │
│    ⚠️ 3件: 領収書なし（5千円以上）→ 差し戻し    │
│    ❓ 1件: 接待交際費 → 社長承認が必要          │
│                                               │
│ ■ 適用したルール:                               │
│ - 「5,000円以上は領収書必須」(confidence: 0.95)  │
│ - 「接待交際費は社長承認」(confidence: 0.95)     │
│ - 「インボイス登録番号チェック」(confidence: 0.95) │
│                                               │
│ ■ 参照したナレッジ:                             │
│ - 経費精算フロー (common/経理)                  │
│ - 消費税・インボイス制度対応 (common/経理)       │
│                                               │
│ ■ 信頼度: 92% (Level 2)  連続成功: 28回         │
│                                               │
│ [✅ 一括承認] [📝 個別確認] [❌ 却下]            │
│                                               │
└──────────────────────────────────────────────┘
```

G. パラメータ化オンボーディング

G.1 テンプレートの `{{placeholder}}` を埋めるフロー

```
■ オンボーディング対話（初回セットアップ時）

  シャチョツー: 「御社の経費精算ルールを確認させてください」

  Q1: 「経費精算の締め日は何日ですか？」
      → {{expense_deadline_day}} = 10  [翌月10日]

  Q2: 「領収書が必要になる金額の基準は？」
      → {{expense_receipt_threshold}} = 5000  [5,000円以上]

  Q3: 「承認フローを教えてください」
      → {{expense_approval_flow}} = "5万円以下: 部門長 / 5万超: 社長"

  Q4: 「出張時の日当はいくらですか？」
      → {{daily_allowance_day}} = 2000
      → {{daily_allowance_stay}} = 3000
      → {{hotel_limit}} = 10000

  → knowledge_items の content 内の {{...}} を実値で置換
  → 会社専用のナレッジとして確定
  → この瞬間から「経費精算AI社員」がLevel 0で起動可能
```

G.2 対話型パラメータ収集のロジック

```
class OnboardingEngine:
    """テンプレートのplaceholderを対話で埋めるエンジン"""

    async def generate_questions(self, company_id: str) -> list[OnboardingQuestion]:
        """
        未設定の {{placeholder}} を検出し、
        自然な日本語の質問に変換する。
        """
        # knowledge_items から {{...}} パターンを抽出
        items = await self._get_template_items(company_id)
        placeholders = self._extract_placeholders(items)

        # LLMで placeholder → 自然な質問に変換
        questions = []
        for ph in placeholders:
            q = await self._llm_generate_question(
                placeholder=ph.name,
                context=ph.surrounding_text,
                department=ph.department,
            )
            questions.append(q)

        # 重要度順にソート（法的義務のあるものが先）
        questions.sort(key=lambda q: q.priority, reverse=True)
        return questions

    async def apply_answer(
        self, company_id: str, placeholder: str, value: str
    ) -> int:
        """回答をナレッジに反映する。影響を受けたアイテム数を返す。"""
        updated = await self._replace_placeholder(company_id, placeholder, value)
        return updated
```

H. DB追加テーブル

```
-- BPO AI社員の定義
CREATE TABLE bpo_agents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    name TEXT NOT NULL,                          -- 「経費精算AI社員」
    department TEXT NOT NULL,                     -- 「経理・財務」
    job_description TEXT NOT NULL,                -- LLM生成の役割説明
    rules JSONB DEFAULT '[]',                    -- 適用ルール一覧
    tool_ids UUID[] DEFAULT '{}',                -- tool_connections の参照
    source_knowledge_ids UUID[] DEFAULT '{}',    -- 元になった knowledge_items
    current_level INT DEFAULT 0,                 -- 0-4
    trust_score JSONB DEFAULT '{}',              -- 信頼度スコア詳細
    consecutive_successes INT DEFAULT 0,
    total_executions INT DEFAULT 0,
    approved_count INT DEFAULT 0,
    rejected_count INT DEFAULT 0,
    modified_count INT DEFAULT 0,
    is_active BOOLEAN DEFAULT false,             -- 社長が有効化
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- BPOタスク（発見されたタスク）
CREATE TABLE bpo_tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    agent_id UUID NOT NULL REFERENCES bpo_agents(id),
    trigger_type TEXT NOT NULL,                   -- schedule / event / condition /
proactive
    trigger_source TEXT,                          -- cron式 / SaaSイベント名 / KI_id
    title TEXT NOT NULL,
    description TEXT,
    input_data JSONB DEFAULT '{}',               -- SaaSから取得したデータ
    output_data JSONB DEFAULT '{}',              -- 実行結果
    status TEXT DEFAULT 'pending',               --
pending/executing/draft/approval/approved/rejected/completed
    execution_level INT NOT NULL,                -- 実行時のレベル
    audit_card JSONB DEFAULT '{}',               -- 監査カード（判断プロセスの可視化）
    created_at TIMESTAMPTZ DEFAULT now(),
    completed_at TIMESTAMPTZ
);

-- BPOスケジュール（Cronトリガー）
CREATE TABLE bpo_schedules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    agent_id UUID NOT NULL REFERENCES bpo_agents(id),
    cron_expression TEXT NOT NULL,               -- "0 9 25 * *"
```

```
    description TEXT,                           -- 「毎月25日9時: 勤怠締め処理」
    next_run_at TIMESTAMPTZ,
    last_run_at TIMESTAMPTZ,
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- パラメータ（テンプレートの{{placeholder}}の実値）
CREATE TABLE company_parameters (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    param_key TEXT NOT NULL,                    -- "expense_deadline_day"
    param_value TEXT NOT NULL,                  -- "10"
    param_type TEXT DEFAULT 'text',             -- text / number / date / json
    department TEXT,                            -- どの部門のパラメータか
    source TEXT DEFAULT 'onboarding',           -- onboarding / manual / inferred
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(company_id, param_key)
);

-- 全テーブルにRLS
ALTER TABLE bpo_agents ENABLE ROW LEVEL SECURITY;
ALTER TABLE bpo_tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE bpo_schedules ENABLE ROW LEVEL SECURITY;
ALTER TABLE company_parameters ENABLE ROW LEVEL SECURITY;
```

I. 実装フェーズ

Phase	内容	期間
Phase 0.5	パラメータ化オンボーディング [G] + company_parameters テーブル	1週間
Phase 1.5a	Agent Factory [A] — ナレッジ→AI社員ロール自動生成	2週間
Phase 1.5b	Task Discovery [B] — スケジュール+イベントトリガー	2週間
Phase 1.5c	Executor Level 0-2 [C] — 通知→収集→下書き	2週間
Phase 1.5d	Audit Card [F] + 承認UI	1週間
Phase 2a	Feedback Loop [D] — 却下→ナレッジ更新	2週間
Phase 2b	Trust Scorer [E] + Level 3-4 昇格	1週間
Phase 2c	knowledge_relations triggers チェーンからのワークフロー発見	2週間

J. 既存設計書との整合

設計書	本設計との関係
`b_02_BPO編.md` §4.1 ワークフローマイニング	→ [B] Task Discovery の条件トリガーで実装
`b_02_BPO編.md` §4.2 エージェント分化	→ [A] Agent Factory で自動生成
`b_02_BPO編.md` §4.3 BPO品質保証マトリクス	→ [E] Trust Scorer + [F] Audit Card
`b_02_BPO編.md` Level 0-4 定義	→ [C] Executor のレベル別処理
`b_02_BPO編.md` 棄却学習	→ [D] Feedback Loop
`c_02_プロダクト設計.md` proactive_proposals	→ [D] のナレッジ更新提案で使用
`c_02_プロダクト設計.md` tool_connections	→ [A] のツールバインディングで参照
`c_03_実装計画.md` Phase 4-5	→ 本設計の Phase 1.5-2 に対応